

Cursors with SQL Server Part 2

By Don Schlichting

Introduction

In the preceding article, cursor use and basic syntax were covered. A select cursor was created demonstrating the keywords DECLARE, OPEN, FETCH, @@FETCH_STATUS, and DEALLOCATE. These keywords guide basic cursor execution by assigning a TSQL statement to the cursor, moving records into the cursor, retrieving a specific record from the cursor, working with retrieved record, and then closing the cursor. The sample cursor demonstrated that SQL could act on one record at a time, as opposed to its usual method of acting on sets of data. In this article, the sample cursor will be expanded upon to include data modification and record positioning.

Following is the sample cursor from the first article. This example demonstrates the minimum statements required to create and use a cursor.

```
USE pubs
GO

DECLARE get_price CURSOR FOR
    SELECT price FROM titles

OPEN get_price

FETCH NEXT FROM get_price

WHILE @@FETCH_STATUS = 0
    FETCH NEXT FROM get_price

CLOSE get_price
DEALLOCATE get_price
```

SELECT INTO

The purpose of the this cursor was to lay the ground work for a statement that will change books priced under \$20 to be raised by 10%, while books currently \$20 or more, will be raised 5%. The above cursor will now be modified so prices returned from titles under twenty dollars will print "Under 20", while titles twenty dollars or more will print the price. Modify the cursors to:

```
DECLARE @price money
DECLARE @get_price CURSOR

SET @get_price = CURSOR FOR
    SELECT price FROM titles

OPEN @get_price
```

```

FETCH NEXT FROM @get_price INTO @price

WHILE (@@FETCH_STATUS = 0)
BEGIN
    IF @Price < 20
        SELECT 'Under 20'
    ELSE
        SELECT @Price

    FETCH NEXT FROM @get_price INTO @price
END

CLOSE @get_price
DEALLOCATE @get_price

```

One of the first differences to notice is the use of variables for the cursor and the price. Now that the cursor is a variable, it is SET to a SQL statement. The SQL statement itself has not changed. The cursor is opened like before, but the FETCH will move the field price into the variable @price. If more than one column were needed, say price and title, we would declare two variables, then SELECT INTO both as follows:

```

DECLARE @Price money, @Title varchar(50), @get_price CURSOR

SET @get_price = CURSOR FOR
    SELECT Price, Title FROM titles

OPEN @get_price

FETCH NEXT FROM @get_price INTO @Price, @Title

```

The WHILE loop is still controlled with @@FETCH_STATUS, the use of variables does not alter this. Once fetched, @price can be used like a normal variable as demonstrated by "*IF @Price < 20*". Because the WHILE loop has more than one action line, it uses BEGIN and END tags. Once we are done working with this record, the next is fetched. At end of file, the loop exits and the cursor is closed and destroyed.

Running the cursor will produce the expected list of "Under," price, and nulls, depending on the value found in the titles table.

	(No column name)
1	Under 20
	(No column name)
1	Under 20
	(No column name)
1	NULL
	(No column name)
1	22.9500
	(No column name)
1	20.0000

WHERE CURRENT OF

With the cursor running as expected, it can be easily modified to update the values rather than display them. Modify the cursor to include the following update statements:

```
DECLARE @price money
DECLARE @get_price CURSOR

SET @get_price = CURSOR FOR
    SELECT price FROM titles

OPEN @get_price

FETCH NEXT FROM @get_price INTO @price

SELECT price FROM titles

WHILE (@@FETCH_STATUS = 0)
BEGIN
    IF @Price < 20
        UPDATE titles SET price =
            ➡(@price + (@price * .1))
        WHERE CURRENT OF @get_price
    ELSE
        UPDATE titles SET price =
            ➡(@price + (@price * .05))
        WHERE CURRENT OF @get_price

    FETCH NEXT FROM @get_price INTO @price
END

SELECT price FROM titles

CLOSE @get_price
DEALLOCATE @get_price
```

The selects show the expected results. Prices over twenty dollars have been raised by five percent while prices fewer than twenty dollars have been raised ten percent.

	price
1	19.9900
2	11.9500
3	2.9900
4	19.9900
5	19.9900
6	2.9900
7	NULL
8	22.9500

	price
1	21.9890
2	13.1450
3	3.2890
4	21.9890
5	21.9890
6	3.2890
7	NULL
8	24.0975

In the TSQL, the two new lines:

```
SELECT price FROM titles
```

were added only to show our changes. They are not needed for the update. The new key phrase "WHERE CURRENT OF" is now being used. The phrase tells SQL to work with the record at the current cursor position. Behind the scenes, SQL is using a dynamic cursor with optimistic locking on our cursor.

Cursor Types

There are three types of cursors, DYNAMIC, STATIC, and KEYSET. Dynamic cursors will show changes made on the base table as you scroll through the cursor. Static cursors copy the base table, "titles" in our case, to the tempdb. The cursor then reads from the tempdb, so any changes happening on the base table will not be reflected in the cursors scrolling. Keysets are in between Dynamic and Static cursors. A keyset will copy the base table's selected records keys into the tempdb, so the cursor will select the rows from the tempdb, but the data from the base table. So change to the base table will be seen, but new record inserts will not be.

Lock Types

There are also three types of locks for cursors, READ ONLY, SCROLL LOCKS, and OPTIMISTIC. A Read Only lock prevents any updates. It also cannot be used with the WHERE CURRENT OF statement. A scroll lock issues a hard lock on the underlining records. Any updates made by the cursor are guaranteed to succeed. Optimistic locking will look at

the underlying record for any changes since the cursor fetched it, if the base tables record has changed, your update will fail. If not, the cursors update will succeed.

Optimization

The first rule of cursor optimization is not to use them if possible. In our above example, two update statements could have been used rather than a cursor. One statement would select books priced over twenty dollars, while the other selects books priced fewer than twenty dollars. This will usually out perform a cursor. If you are not going to be doing DML, use a read only or fast forward cursor rather than a dynamic scrollable cursor. Also, close and destroy the cursor when finished with it.

Conclusion

Sometimes the usual TSQL method of working with sets of records does not fit the application. If so, a cursor can be used to move SQL on a record-by-record basis. Cursors support a wide and useful array of commands to control everything from locks to visibility. Used in their correct format, cursors are reliable and invaluable.